Aravindh Varadharaj

Full-Stack Engineer · Go · React · TypeScript · Distributed Systems · Data Pipelines

Tiruppur, Tamil Nadu, India · open to relocation, on-site or remote

aravindhvaradha@gmail.com | github.com/Aravindh97 | LinkedIn | aravindh97.github.io/resume

10 projects
each with a **live demo**: Go compiled to WebAssembly with a **React + TypeScript** frontend, tests, CI and measured results.
Try them at aravindh97.github.io/resume

SUMMARY

Builder at Amazon since 2021 who moved from digital marketing into engineering by automating my team's workflows. I build full-stack systems, Go services with React + TypeScript frontends, and prove they hold up: concurrency under load, exactly-once money movement, crash and outage tests, offline sync, streaming pipelines and performance work, each with tests, CI and measured results against a baseline.

SKILLS

Go	goroutines, channels, context, sync, net/http, generics, WebAssembly, race detector, benchmarks and profiling
Frontend	React, TypeScript, Vite, Vitest; TypeScript types generated from Go structs
Data	PostgreSQL (transactions, row locking, SKIP LOCKED, advisory locks, partial unique indexes, logical replication, pgvector), OpenSearch, Redis basics
Reliability	idempotency keys, retries with backoff, dead-letter states, Temporal workflows, kill -9 and outage testing, watermarks and backpressure
Distributed systems	Raft, CRDTs, version vectors, hybrid logical clocks, CDC
Quality	evals and backtests with baselines, chance and held-out data, integration tests on Docker
AI coding tools	Kiro, Claude Code and Copilot daily; concurrency, SQL and retry logic reviewed by hand

PERSONAL PROJECTS

Built in my own time to learn; not affiliated with my employer. Live demos and source at github.com/Aravindh97.

fairdrop · Ticketing for High-Demand Releases

Live demo · GitHub

Go | PostgreSQL | drand public randomness | React + TypeScript frontend

- Seat holds claim exactly the requested seats or none in one locked UPDATE (fixed lock order, SKIP LOCKED); payments are idempotent by key; families are seated together.
- 50,000 simulated buyers with timeouts, retries, double clicks and abandoned holds: 0 seats sold twice in 3 runs vs 156,874 extra bookings for the naive version; charges always equal bookings plus refunds.
- Queue order comes from a future drand round committed before registration closes, so any buyer can verify it; swapped positions and late entries are detected.

paisa · Exactly-Once Double-Entry Ledger

Live demo · GitHub

Go | PostgreSQL | React + TypeScript frontend

- Idempotency key, postings and balances written in one transaction; retries return the stored outcome, including rejections; reusing a key for a different request is refused.
- kill -9 the payment process 20 times during 50,000 payments: books balanced after every kill and final balances of all accounts matched a clean run exactly.
- Reconciliation found 477 of 477 planted bank-file mismatches and fixed them idempotently; sharding the hot fee account raised throughput 6.8x (437 to 2,981 payments/s).

Granth · Document Ingestion & Hybrid Retrieval for Agents

Live demo · GitHub

Go | Temporal | OpenSearch | PostgreSQL | Tesseract OCR | MCP | React + TypeScript frontend

- One Temporal workflow per document (fetch, parse, OCR fallback, chunk, embed, index). Activities pass sha256 blob references and chunk ids are deterministic, so retries and replays never create duplicates.
- Corrupt, unsupported or oversized files fail once and go to a Postgres dead-letter ledger with the failing step. Transient failures retry with backoff for up to an hour; an embedding cache stops retries from paying twice.
- Stopped OpenSearch mid-ingest in a live test: the index step retried 7+ times and the document indexed after restart with no duplicate chunks.

orbit · Satellite Conjunction Screener

Live demo · GitHub

Go | SGP4 | CelesTrak public catalogue | React + TypeScript frontend

- Altitude shells, a spatial hash sized from the maximum closing speed, and a straight-line prefilter before golden-section refinement: checks 0.01% of pairs, found 190 of 190 encounters an exhaustive brute force found.
- 16,763 objects over 6 days in 3 min 49 s on 8 cores (3.8 M object-steps/s); fixed an epoch bug in the upstream SGP4 port (median 3.6 km error), now within 1 mm of reference.

Smriti · Long-Term Memory for LLM Agents

Live demo · GitHub

Go · PostgreSQL + pgvector · Temporal · MCP · React + TypeScript frontend

- User, team and org memories with one access rule applied in every query. Org writes need explicit permission, so one user's chat cannot change what every agent believes.
- Each fact is added, confirmed or superseded instead of appended. An advisory lock plus a partial unique index keep one current value per subject (tested with 20 concurrent writers); replaying a conversation is a no-op.
- Bitemporal rows keep history and source message, so agents can answer both "what is true now" and "what did we believe in February".

Sutra · Enterprise Connectors, Ontology & Entity Resolution

Live demo · GitHub

Go · PostgreSQL logical replication (pgoutput) · REST · YAML ontology · MCP · React + TypeScript frontend

- CDC from Postgres with an exported-snapshot initial load and commit-time checkpoints. After a kill -9 mid-stream and restart, source and warehouse matched exactly (md5 over every row).
- Incremental CRM sync with keyset pagination, Retry-After handling and checkpoints written in the same transaction as each page.
- Entity resolution with GSTIN checksum validation, PAN derived from GSTIN, +91 phone normalisation and transliteration-aware names (Sri/Shree, Lakshmi/Laxmi). Two different PANs can never merge; unsure pairs go to review.

relief · Offline-First Sync for Disaster Relief Teams

Live demo · GitHub

Go · CRDTs · hybrid logical clocks · React + TypeScript frontend

- Counters for stock, add-wins needs and HLC last-writer-wins fields over an op log; version vectors send 3.3% of the data a full copy would; works over USB bundles.
- 10,000 randomised runs with partitions, relays, duplicates and clock skew: every device converged with 0 rule violations; last-writer-wins sync lost data in all 10,000.

aqcal · Low-Cost Air Sensor Calibration and Fault Detection

Live demo · GitHub

Go streaming pipeline · real Berlin sensor and UBA reference data · React + TypeScript frontend

- Sharded aggregation with watermarks and bounded channels: 6.9 M readings/s on 8 shards.
- Per-sensor calibration cut hourly PM2.5 error 52% (4.81 to 2.31 ug/m3) on held-out days; detectors caught 298 of 300 injected faults.

foodwatch · Food-Stress Early Warning (Somalia)

Live demo · GitHub

Go · CHIRPS rainfall · WFP prices · IPC classifications · React + TypeScript frontend

- Backtested against IPC crisis classifications with no lookahead: the rainfall signal flagged 20 of 26 regional crisis onsets (10.7 expected by chance) a median 21.6 weeks early.
- Cross-checked rainfall sources: NASA POWER called the failed 2021 Deyr season wet, so the backtest uses CHIRPS.

Game: Pallanguzhi (Tamil mancala) against a Go alpha-beta search AI that explains its moves, with a React + TypeScript board in the browser; a reproducible 2,800-game parallel benchmark showed the second player has a large advantage, so results are reported by seat.

Earlier projects: In github.com/Aravindh97/aravindh-impact-portfolio: OpenSchool Enterprise (Python/FastAPI learning platform with role contracts and auditable review state), FamineLens (sourced historical evidence with an uncertainty-aware scenario model), Kindred (privacy-aware animal welfare network with a human-review queue), EcoSignal (Go RF power-plan optimiser under a coverage constraint), Nuclear Legacy & Civil Safety (Go observatory).

EXPERIENCE

Sr. Digital Marketing Associate, Amazon 2021 to Present

- Built and automated internal tools and workflows for my team, replacing repetitive manual tasks with reliable, reusable solutions.
- Worked with stakeholders to find process gaps, then designed, built and supported automation that the team adopted in daily work.
- Grew from a marketing role into hands-on development, and built Go and Python backend systems outside my core responsibilities.

EDUCATION

Bachelor of Engineering (B.E.) in Mechanical Engineering